

الدليل الكامل خطوة بخطوة

BugIt

وكيل ضمان الجودة لإنشاء تذاكر الخلل وإرسالها إلى نظام التتبع عبر VS Code

الإعداد لأول مرة والاستخدام اليومي — دليل تفصيلي يشرح كل نقرة، ومكتوب لمن لم يسبق لهم القيام بأي شيء مماثل. لا تحتاج إلى خلفية تقنية. اتبعه بالترتيب، وستتمكن اليوم من إنشاء تذاكر خلل واضحة ومنظمة وإرسالها إلى نظام التتبع.

يعمل في: VS Code — Windows (الأساسي)، macOS، وLinux

تنويه بشأن الترجمة.

تُرجم هذا المستند آليًا ولم يخضع لمراجعة متحدثين أصليين باللغة. النسخة الإنجليزية هي المرجع المعتمد: وعند وجود أي اختلاف يُعتقد بالنص الإنجليزي. وللإطلاع على أدق صياغة وأحدثها، يُرجى الرجوع إلى المستند الإنجليزي.

محتويات الدليل

نقّذ الجزء 1 مرة واحدة وبالترتيب. بعد ذلك، يمكنك الرجوع إلى أي جزء كلما احتجت إليه. يجب قسم استكشاف الأخطاء وإصلاحها والأسئلة الشائعة قرب نهاية الدليل عن أكثر الأسئلة شيوعًا؛ يُرجى الاطلاع عليه قبل مراسلة الدعم عبر البريد الإلكتروني.

الجزء 1 • الإعداد (نقّذ هذا مرة واحدة)	الخطوة 7 — شغل <code>Begin setup</code>
الخطوة 0 — احصل على GitHub Copilot	الخطوة 8 — صل نظام التنبّع الخاص بك
الخطوة 1 — ثبّت VS Code	الخطوة 9 — احفظ بيانات تسجيل الدخول
الخطوة 2 — فك ضغط ملف BugIt الذي نزّلته	الخطوة 10 — تأكد من جاهزيتك
الخطوة 3 — افتح BugIt في VS Code	الجزء 2 • الاستخدام اليومي
الخطوة 4 — اختر وكيل BugIt في الدردشة	الجزء 3 • خصّصه ليناسبك
الخطوة 5 — فعّل ترخيصك	الجزء 4 • التحديثات والنسخ الاحتياطية والسلامة
الخطوة 6 — اقبل الشروط (مرة واحدة)	الجزء 5 • استكشاف الأخطاء وإصلاحها والأسئلة الشائعة
	الجزء 6 • الترخيص والدعم

القاعدة الوحيدة التي تحافظ على سلامتك

لا ينشئ BugIt أبدًا أي تذكّرة ولا يرسلها من دون أن يعرض عليك معاينة ويحصل أولًا على موافقتك المكتوبة `yes`. أنت المتحكّم دائمًا. هل تريد التدرّب بلا أي مخاطرة؟ اكتب `dry run`، وسيكتب التقرير كاملاً من دون إنشاء تذكّرة أو إرسالها.

الإعداد لأول مرة مقابل الاستخدام اليومي

لن تنقّذ الجزء 1 كاملاً إلا مرة واحدة. بعد ذلك، يصبح فتح BugIt سريعًا، ولا تتطلب التحديثات سوى أمر واحد (`Update`). راجع الجزء 4.

الجزء 1 • الإعداد — نقّذ هذا مرة واحدة

يستغرق نحو 15 دقيقة. نقّذ الخطوات بالترتيب؛ فكل خطوة تعتمد على سابقتها. لا تتجاوز أي خطوة.

0 حصل على GitHub Copilot

لا يمكنك استخدام Bugt من دونه

Bugt هو «السائق»، وGitHub Copilot هو «المحرك»؛ أي الذكاء الاصطناعي الذي يكتب التقارير فعليًا. إذا لم يكن Copilot يعمل لديك داخل VS Code، فلن يفيدك أي مما يلي. أعدّه أولًا.

ما هو؟ GitHub Copilot مساعد يعمل بالذكاء الاصطناعي من GitHub، المملوكة لشركة Microsoft. يوفر مستوى مجانيًا للاستخدام الخفيف ومستوى مدفوعًا للاستخدام المكثف (حوالي \$10 شهريًا). ستبته في الخطوة 1 وتسجل الدخول.

1. ستحتاج إلى **حساب GitHub**. إذا لم يكن لديك حساب، فأنشئ حسابًا مجانيًا على github.com/signup.
2. يكفي المستوى المجاني من Copilot لتجربة Bugt. إذا كنت تسجل بلاغات أخطاء طوال اليوم، فإن خطة **Copilot Pro** المدفوعة تستحق تكلفتها؛ ويمكنك الترقية في أي وقت على github.com/features/copilot.
3. ستفعل Copilot فعليًا داخل VS Code في الخطوة 1؛ ولا تحتاج إلى فعل أي شيء آخر هنا الآن.

1 ثبت VS Code

VS Code تطبيق مجاني من Microsoft. وهو «النافذة» التي يعمل Bugt داخلها.

1. افتح متصفح الويب وانتقل إلى code.visualstudio.com.
2. انقر زر **Download** الأزرق الكبير؛ فهو يكتشف نظامك تلقائيًا.
3. افتح الملف الذي نزلته وثبته؛ اقبل **جميع الخيارات الافتراضية**، وواصل النقر على **Next/Install**.
4. افتح VS Code بعد تثبيته.

فعل GitHub Copilot داخل VS Code

1. في أقصى يسار VS Code، انقر أيقونة **Extensions**، التي تبدو كأربعة مربعات صغيرة.
2. في مربع البحث، اكتب **GitHub Copilot**.
3. انقر **Install** على **GitHub Copilot**، وكذلك على **GitHub Copilot Chat** إذا ظهر.
4. ستظهر مطالبة تطلب منك اختيار **Sign in to GitHub**؛ انقر هذا الخيار وسجل الدخول إلى حسابك على GitHub في نافذة المتصفح التي تفتح.

✅ **ستعرف أنه نجح عندما:** تظهر أيقونة Copilot صغيرة أسفل VS Code، وتفتح لوحة **Chat**؛ وهي أيقونة فقاعة الكلام على اليسار، أو يمكنك الضغط على **Ctrl+Alt+I**.

2 فك ضغط ملف BugIt الذي نزلته

- تضمّن شراؤك ملفًا بامتداد `.zip` ، مثل `bugit-qa-agent.zip` . عليك فك ضغطه؛ فلن يعمل إذا شغلته وهو مضغوط.
1. ابحث عن ملف `.zip` ، وعادةً ما يكون في مجلد `Downloads` .
 2. انقر عليه بزر الفأرة الأيمن ← اختر `Extract All...` في Windows ، أو انقر عليه نقرًا مزدوجًا في Mac.
 3. اختر مكانًا بسيطًا مثل مجلد `Documents` ، ثم انقر `Extract` .
 4. سيصبح لديك الآن مجلد **BugIt** يحتوي على ملفات كثيرة. تذكّر مكانه.

لا تضعه في OneDrive أو في مجلد متزامن

قد تسبب مجلدات المزامنة السحابية تعارضات في الملفات. مجلد `Documents` أو `Desktop` مناسب تمامًا. كذلك لا تغيّر أسماء الملفات داخله؛ اترك المجلد كما هو.

3 افتح BugIt في VS Code

1. في VS Code ، انقر `File → Open Folder` من القائمة العلوية اليسرى.
2. انتقل إلى المكان الذي فككت فيه ضغط BugIt ، مثل `Documents` .
3. انقر مجلد **BugIt** مرة واحدة لتحديده، ثم انقر `Select Folder` .
4. إذا سألك VS Code: `Do you trust the authors of the files in this folder?` ، فانقر `Yes, I trust the authors` .

✓ ستعرف أنه نجح عندما: ترى ملفات BugIt مدرجة على الجانب الأيسر من VS Code ، ومنها مجلدات مثل `tools` و `github` . وملفات مثل `README.md` .

4 اختر وكيل BugIt في الدردشة

1. افتح لوحة Copilot Chat؛ انقر أيقونة فقاعة الكلام على اليسار، أو اضغط `Ctrl+Alt+I` في Windows أو `Cmd+Alt+I` في Mac.
2. توجد أعلى مربع الدردشة قائمة منسدلة صغيرة للأوضاع، وقد تعرض `Ask` أو `Agent` .
3. انقر عليها واختر **BugIt QA Agent** من القائمة.

ألا ترى BugIt QA Agent في القائمة؟

تأكد من أنك فتحت مجلد BugIt في الخطوة 3، لا مجلدًا مختلفًا؛ فالوكيل يُحمّل من داخله. أغلق المجلد وأعد فتحه إذا لزم الأمر.

5 فعل ترخيصك

كل ما تحتاج إليه هو حساب BugIt؛ فلا يوجد مفتاح ترخيص لتنسخه أو تلصقه. يجري التفعيل في متصفحك.

1. في مربع الدردشة، اكتب **hi** واضغط **Enter**.
2. اكتب **Activate**. يفتح BugIt موقع **BugIt Portal** في متصفحك. سجّل الدخول إلى حسابك الشخصي، واختر استحقاق **Solo** أو **Team**، ووافق على هذا الجهاز. ثم عدّ إلى VS Code؛ يكمل BugIt التفويض تلقائيًا. تبقى كلمة مرورك في المتصفح ولا تُدخل أبدًا في VS Code.

✓ ستعرف أنه نجح عندما: يرد BugIt بالنص "✓ Activated – licensed to [your name/email]."

حافظ على خصوصية حسابك

حسابك والاستحقاق المرتبط به خاصان بك. لا تشاركهما أو تنشرهما أو تعيد بيعهما؛ فقد يلغى الوصول المشترك. يسجّل كل عضو في **Team** الدخول بحسابه الخاص؛ فلا يوجد مفتاح مشترك ولا بيانات دخول مشتركة. ولا بأس بالانتقال إلى كمبيوتر جديد لاحقًا؛ راجع الأسئلة الشائعة.

6 اقبل الشروط (مرة واحدة)

في المرة الأولى، يعرض BugIt ملخصًا قصيرًا ويطلب منك القبول قبل أن يفعل أي شيء آخر.

1. اقرأ الملخص القصير الذي يعرضه: تراجع كل تذكرة قبل إنشائها وإرسالها؛ وتقع مسؤولية سلامة مشروعك عليك؛ وتُعد إجراءات الحماية وسائل مساعدة وليست ضمانات. ولأن نموذج الذكاء الاصطناعي الذي يجيبك في Copilot هو الذي يتبع هذه الإجراءات، فقد يتجاوز نموذج أخف أو مجاني خطوة أحيانًا، أو يحتاج إلى تكرار أمر، في حين لا يحتاج نموذج أقوى إلى ذلك.
2. أجب **I agree** واضغط **Enter**.

✓ ستعرف أنه نجح عندما: يؤكد BugIt ذلك وينتقل إلى الإعداد. لن تُسأل عن هذا إلا مرة واحدة؛ فهو يتذكر.

7 شغل **Begin setup**

يُهيئ BugIt نفسه الآن من خلال طرح أسئلة عليك بلغة واضحة. لن تحتاج أبدًا إلى تعديل ملف إعدادات يدويًا.

أنت تكتب

Begin setup

سيطرح أسئلة قصيرة ولن يفعل إلا ما تختاره. وأهم سؤال يتعلق بنظام تتبّع العيوب الذي تستخدمه:

يسأل... ما الذي تجيب به

أي نظام لتتبع المشكلات تستخدم؟ (اختر واحدًا — مطلوب)	المكان الذي يحتفظ فيه فريقك بتقارير الخلل. يحصل Jira و Azure DevOps على إعدادات موجهة يتضمن تعيينًا للحقول جرى اختبارها، وتُضبط درجة الخطورة/الأولوية تلقائيًا بما يتوافق مع تقريرك. يستخدم Atlassian Jira MCP مع تسجيل الدخول عبر المتصفح؛ ويستخدم Azure DevOps خادم MCP البعيد من Microsoft والمحدد النطاق على مستوى المؤسسة (معاينة عامة)، مع تسجيل الدخول عبر المتصفح أيضًا. تستخدم Linear و GitHub و Sentry و Notion نقطة نهاية معروفة يُعدها BugIt ويتحقق منها مباشرةً على حسابك؛ تعامل معها بوصفها تجريبية حتى تنجح تلك الفحوصات. أما الأنظمة الأخرى (GitLab و Shortcut و YouTrack و Bugzilla و ClickUp و Asana و Trello)، فتتصل من خلال خادم MCP متوافق خاص بك، ويساعدك BugIt على إعدادها. اختر النظام الذي تستخدمه بالفعل.
أداة لتتبع الأعطال؟ (اختياري)	تستخدم Sentry نقطة نهاية معروفة يتحقق منها BugIt مباشرةً على حسابك (تجريبية)؛ أما BugSnag و Crashlytics و Raygun و Rollbar و App Center و Datadog، فتتصل من خلال خادم MCP متوافق خاص بك، وذلك فقط إذا كان فريقك يستخدم إحداها.
إدارة الاختبارات؟ (اختياري)	Qase, Zephyr, Xray, TestRail.
نشر تقارير الخلل في الدردشة؟ (اختياري)	Slack أو Microsoft Teams أو Discord أو البريد الإلكتروني. الصق عنوان webhook (أو تفاصيل خادم البريد للبريد الإلكتروني)، وسيُرسل BugIt رسالة اختبار فعلية، ولن يحفظ الإعداد إلا بعد وصولها. تختار لكل قناة الأحداث التي تريدها، وهي filed و commented و attached ، إلى جانب الحد الأدنى لدرجة الخطورة، بحيث لا تتلقى قناة الإصدار مثلًا إلا بلاغات High و Critical. ولا يؤدي فشل الإرسال أبدًا إلى فشل إنشاء التذكرة.
المواصفات / المعرفة؟ (اختياري)	Confluence أو Notion أو SharePoint أو Google Docs أو مجلد محلي يضم ملفات Markdown داخل هذا المستودع.
اللغات	لغتك الأساسية (الافتراضية هي English). ثم يطرح سؤالًا بسيطًا بنعم أو لا عما إذا كنت تنشئ البلاغات بلغة ثانية أيضًا؛ معظم الفرق تستخدم لغة واحدة فقط.
الميزات المتقدمة؟ (اختياري)	أدوات إضافية مثل ملخص الفرز والتراجع والتصدير دون اتصال؛ راجع الجزء 2. جميعها اختيارية.

نصيحة: ابدأ من قالب

إذا كان عملك يندرج ضمن فئة محددة، فاكتب `game preset.py` (وكذلك `web-saas` و `mobile` و `enterprise`) لملء فئات وإعدادات مناسبة مسبقًا. ويمكنك دائمًا تغييرها لاحقًا.

هل توقف الإعداد مؤقتًا أو انقطع؟ ما عليك سوى قول الأمر مرة أخرى

إذا توقف `Begin setup` في منتصف العملية، سواء لأنك أغلقت VS Code أو لأنه توقف مؤقتًا فحسب، فإن كتابة `Begin setup` مرة أخرى تستأنف العملية من الموضع الذي توقفت عنده. ولن يجعلك تعيد الإجابة عن أسئلة سبق أن أجبت عنها. ولتغيير شيء عمدًا في وقت لاحق، مثل إضافة نظام تتبع أو استبدال اختيار، ما عليك سوى قول ذلك، مثلًا: `"add a tracker"`، وسيعيد فتح ذلك الجزء من أداة الاختيار.

يتواصل نظام تتبع العيوب مع BugIt من خلال جسر صغير (اسمه التقني هو `MCP server`). يُعدّ BugIt هذا الجسر لك؛ وما عليك سوى الإجابة عن الأسئلة والنقر على زر واحد. ولن تعدّل أي ملفات.

8a • دع BugIt يهيئ الجسر

1. أثناء الإعداد، يعرض BugIt توصيل نظام تتبع العيوب. وبالنسبة إلى Jira و Confluence (Atlassian Rovo)، فهو يعرف خطوات الاتصال الموجهة؛ أما Sentry و GitHub و Linear و Notion، فيعرف نقطة النهاية القياسية ويتحقق منها مباشرةً على حسابك. أكد عندما يطلب منك ذلك، وسيجري الفحوصات قبل اعتمادك عليها. تعامل مع هذه الأنظمة بوصفها تجريبية حتى تنجح الفحوصات.
2. بالنسبة إلى الأدوات المستضافة ذاتيًا أو التي توفرها مؤسستك، سيخبرك في جملة واحدة بمكان العثور على عنوان خادم MCP المتوافق، وسيطلب منك لصقه في الدردشة، ثم يكتبه في الإعدادات.

8b • شغل الجسر

1. افتح الملف `.vscode/mcp.json`. من قائمة الملفات على اليسار (انقر عليه مرة واحدة).
2. ابحث داخله عن نص رمادي صغير يقول `Start | More...`؛ ستجده مباشرةً فوق اسم كل خادم. إنه صغير ومن السهل ألا تلاحظه.
3. انقر على `Start` لكل خادم طلب منك BugIt تشغيله (وعادةً ما يكون خادمًا واحدًا لنظام تتبع العيوب).

8c • سجّل الدخول عندما يُطلب منك ذلك

- عندما يتصل الجسر للمرة الأولى، يحتاج إلى التحقق من هويتك. سيحدث أحد الأمرين التاليين:
- ستُفتح نافذة متصفح ← سجّل الدخول إلى نظام تتبع العيوب كالمعتاد (وهذا شائع مع GitHub و Azure DevOps).
 - سيظهر مربع صغير أعلى VS Code ← سيطلب عنوان بريدك الإلكتروني و/أو رمز وصول. الصقه واضغط `Enter`.

من أين أحصل على رمز وصول؟

رمز الوصول يشبه كلمة مرور صالحة لمرة واحدة يمنحك إياها نظام تتبع العيوب. أنشئ رمزًا على موقع النظام، ثم الصقه عندما يطلبه المربع. فيما يلي صفحات الخدمات الشائعة؛ سجّل الدخول بالحساب الذي تستخدمه عادةً:

نظام تتبع العيوب	مكان إنشاء الرمز
Confluence / Jira (Atlassian)	<code>id.atlassian.com/manage-profile/security/api-tokens</code>
GitHub	<code>github.com/settings/tokens</code> (أو سجّل الدخول عبر المتصفح فحسب)
GitLab	<code>gitlab.com/-/user_settings/personal_access_tokens</code>
Sentry	<code>sentry.io → Settings → Account → API → Auth Tokens</code>
Linear	<code>linear.app → Settings → API → Personal API keys</code>
Azure DevOps	رمز <code>Personal Access Token</code> بصلاحيّة <code>Work Items: Read & write</code> من <code>dev.azure.com/YOUR-ORG/_usersSettings/tokens</code>

انسخ الرمز فور ظهوره

لا تعرض معظم الخدمات الرمز الجديد إلا مرة واحدة. انسخه فورًا واحتفظ به في مكان آمن إلى أن يُحفظ في المخزن الآمن على جهازك (الخطوة 9). اختر أطول مدة صلاحية يتيحها نظام التتبع حتى لا تضطر إلى تكرار ذلك قريبًا.

8d • تحقق من اتصاله

بعد تشغيل الجسر، يتغير النص الرمادي إلى **Running** أو تظهر نقطة خضراء. وللتأكد، اكتب في الدردشة:

أنت تكتب

Check status

✓ ستعرف أن العملية نجحت عندما: يعرض Buglt نظام تتبع العيوب بالحالة **connected / healthy**.

يتوقف الجسر عند إغلاق VS Code

في كل مرة تعيد فيها فتح VS Code، انقر على **Start** لخادمتك مرة أخرى داخل **.vscode/mcp.json**. ستظل بيانات تسجيل الدخول محفوظة (الخطوة 9)؛ ما عليك سوى إعادة تشغيل الجسر. هذا سلوك خاص بـ VS Code، وليس مشكلة في Buglt.

9 احفظ بيانات تسجيل الدخول

بعد تسجيل الدخول إلى أحد الجسور (الخطوة 8c)، يتذكر موصل نظام التتبع، أو VS Code نفسه، بيانات تسجيل الدخول في المخزن الآمن المدمج في جهازك (**Windows Credential Manager / macOS Keychain / Linux Secret Service**)، ولذلك لن تحتاج إلى البحث عن الرمز مرة أخرى. لا يحتفظ Buglt بنسخة خاصة به؛ إذ لا يحتوي **config.json** إلا على المؤسسات وعناوين URL، ولا تصل قيم بيانات الاعتماد أبدًا إلى سجل الدردشة أو إلى ملف نصي عادي. لمعرفة الموصلات التي سبق تسجيل الدخول إليها، اكتب:

أنت تكتب

Check saved credentials

سيعرض الموصلات التي تتولى المصادقة، ولن **يعرض أبدًا قيم بيانات الاعتماد**. وإذا نسي أحد الجسور بيانات تسجيل الدخول لاحقًا، فما عليك سوى تسجيل الدخول مرة أخرى عندما يطلب منك ذلك (الخطوة 8c).

10 تأكد من جاهزيتك

اطلب من Buglt التحقق من كل شيء مرة أخرى قبل إنشاء أول تذكرة حقيقية:

أنت تكتب

Check readiness

سيعرض أي شيء لا يزال ناقصًا (وعادةً لا يتجاوز **start your bridge** أو **sign in**). وعندما تصبح جميع المؤشرات خضراء، سترى **"Setup complete." ✓**

وبذلك يكتمل الإعداد — نهائيًا

لن تحتاج إلى تكرار الجزء 1. ومن الآن فصاعدًا، كل ما يلزم لفتح BugIt هو: افتح المجلد ← شغل الجسر من `.vscode/mcp.json` ← اكتب `hi`. ولتغيير أي اختيار لاحقًا، ما عليك سوى كتابة `Begin setup` وذكر ما تريد تغييره.

ألا يتوفر GitHub Copilot؟ يوجد معالج في الطرفية

إذا لم تتمكن من استخدام Copilot، فافتح الطرفية المدمجة (`Terminal → New Terminal`) وشغل `python tools/setup_wizard.py`. سيطرح بضعة أسئلة ويكتب إعداداتك من دون الذكاء الاصطناعي. ويمكنك أيضًا تشغيل BugIt بصورة مستقلة باستخدام مفتاح ذكاء اصطناعي خاص بك؛ راجع الأسئلة الشائعة.

الجزء 2 • الاستخدام اليومي

بعد إتمام الإعداد، ستنجز عملك بالكامل من خلال الدردشة. تحدّث بأسلوب طبيعي، أو استخدم الأمثلة المحددة أدناه. اكتب `help` في أي وقت لعرض القائمة الكاملة.

كتابة تقرير خلل

هذه هي المهمة الرئيسية لـ BugIt. صف الخلل بجملة بسيطة؛ فيكتب التذكرة كاملة، ويملأ الإصدار والفئة تلقائيًا، ويتحقق من وجود تذاكر مكررة، ويعرض عليك معاينة، ثم ينشئها ويرسلها إلى نظام التتبع بعد أن تكتب `yes`.

ما تكتبه

Write a bug report: the app crashes every time I tap Save on iPhone

يُعدّ مسودة تتضمن عناوين، وخطوات إعادة إنتاج الخلل، والنتيجة المتوقعة مقابل النتيجة الفعلية، ودرجة الخطورة، والمنصة. كما يضبط دائمًا حقلي الأولوية ودرجة الخطورة في نظام التتبع ليتطابقا مع التقرير، بدلًا من الاكتفاء بقيمة افتراضية عامة. هل تريد إجراء تغييرات قبل إنشاء التذكرة وإرسالها؟ ما عليك سوى قول ذلك، مثل: «اجعل درجة الخطورة مرتفعة» أو «أضف أنه بدأ في أحدث إصدار».

البلاغات السريعة

بالنسبة إلى الأنماط الشائعة (`crash` و `layout` و `slow` و `missing` و `blocker` وغيرها)، يتجاوز النموذج السريع بعض الأسئلة ويملأ الفئة والأولوية نيابةً عنك، مع استمرار إجراء التحقق الكامل من التذاكر المكررة أولًا.

ما تكتبه (أمثلة)

Quick bug: crash on startup after update

Quick bug: layout button overlaps text on the settings screen

Quick bug: blocker can't continue past the payment step

Quick bug: slow the dashboard takes 20 seconds to load

تقارير التعطل

إذا كنت قد ربطت أداةً لتتبع الأعطال، مثل Sentry، فسينفّذ BugIt كل ما سبق، بالإضافة إلى ربط تقريرك بحادثة التعطل وجلب تتبع المكدس نيابةً عنك.

ما تكتبه

Write a crash bug report: game crashes when opening the map

تعليقات التحقق والإغلاق

بعد اختبار إصلاح، يكتب BugIt تعليقًا منظمًا لنشره على التذكرة. وهو يكتب التعليق، وينشره اختياريًا، لكنه لا يغيّر حالة التذكرة. يظل عليك إغلاق التذكرة أو حلّها أو إعادة فتحها بنفسك في نظام التتبع.

ما تكتبه

ما تحصل عليه

Close #123

يسألك عن السيناريو المقصود، مثل: تأكيد الإصلاح، أو الإغلاق بناءً على طلب، أو الإغلاق لأن السلوك مطابق للتصميم، أو إعادة الفتح؛ ثم يكتب التعليق المناسب.

تعليق «تم تأكيد الإصلاح» مع ملء تفاصيل الإصدار.

Write a verification comment for 1.2.0 on Windows

تعليق «ما زالت المشكلة تحدث»، ثم تعيد فتح التذكرة بنفسك.

Write a reopen comment for 1.2.0 on Windows

الترجمة

ترجم التقارير أو المصطلحات بين اللغات التي أعددتها، باستخدام الصياغات الدقيقة لفريقك من المسرد؛ راجع الجزء 3.

ما تكتبه (أمثلة)

Translate to ja: the save button does nothing on the profile screen
Translate this bug report to English: [paste text]

البحث عن المعلومات: المواصفات والمسرد

إذا كنت قد ربطت المواصفات أو ملأت المسرد، فاطرح على BugIt أسئلةً عن مشروعك.

ما تكتبه (أمثلة)

What is [your term]?
Walk me through the checkout flow
What's new in specs?

اكتشاف التذاكر المكررة

يحدث هذا تلقائيًا قبل إنشاء كل تذكرة وإرسالها. يبحث BugIt في نظام التتبع عن التذاكر المطابقة، حتى إن كانت مصاغة بطريقة مختلفة قليلًا، ويحدّرك حتى لا تقدّم الخلل نفسه مرتين.

الميزات المتقدمة، إذا فعلتها

ما يفعله

اكتب هذا

Triage digest

ملخص فوري للاجتماع اليومي: الأخطاء المفتوحة حسب المجال/الخطورة · مع الإشارة إلى الأقدم.

Export bug to
file

يحفظ التقرير بصيغة Markdown/CSV/JSON بدلاً من إرساله إلى نظام التتبع، وهو خيار ممتاز قبل إعداد نظام التتبع.

Undo last filing

يتراجع عن آخر تذكرة أو تعليق عندما يسمح نظام التتبع بذلك.

(تلقائياً)

علامات SLA على الأخطاء العاجلة · حقول خاصة بكل فئة.

قائمة الأوامر الكاملة

أنت تكتب

help

...يعرض كل أمر داخل الوكيل، في أي وقت.

تدرب بأمان في أي وقت

اكتب `dry run` وسيكتب BugIt التقرير كاملاً من دون إنشاء أي تذكرة أو إرسالها، وهو مثالي للتعلّم أو الاختبار.

الجزء 3 · اجعله خاصًا بك — أضف معرفة مشروعك

يأتي BugIt في الأصل كوكيل عام لضمان الجودة من دون معرفة مسبقة بمشروعك. إليك كيفية تعليمه تفاصيل مشروعك بعد الشراء. لا تحتاج إلى البرمجة؛ فمعظم العمل يتم ببساطة عبر التحدث إليه في الدردشة. كل ما تضيفه يبقى على جهازك.

1 · كلمات فريقك: المسرد

يجعل ذلك الترجمات واستنتاج الفئات أدقّ وأكثر ملاءمةً لمشروعك.

1. افتح الملف `github/glossary/terms.template.md` من قائمة الملفات.

2. املاً الجدول بكلمات فريقك: أسماء الميزات، وأسماء الشاشات، وأسماء العناصر، والاختصارات، وترجماتها إذا كنت تستخدم لغتين.

3. احفظ الملف باستخدام `Ctrl+S`. هذا كل شيء؛ سيستخدم BugIt هذه المصطلحات الدقيقة من الآن فصاعدًا.

اختصار

فعل «البذر التلقائي للمسرد» أثناء `Begin setup`، وسيقترح BugIt مصطلحات مأخوذة من تذاكر الحالية؛ وكل ما عليك فعله هو الموافقة على كل مصطلح.

2 · أسلوب كتابة تقارير الخلل المعتمد لدى فريقك

هل تريد أن تتبع التذاكر تنسيق فريقك بدقة، بما في ذلك أسلوب العناوين، وتسميات الخطورة، والحقول المطلوبة؟ لا تصفه له، بل اعرضه عليه:

1. قل في الدردشة: «طابق أسلوب فريقتي».

2. الصق لقطة شاشة واحدة لتذكرة موجودة في نظام التتبع.

3. يدرسها BugIt محليًا، بعد إزالة أي بيانات شخصية أولًا، ويحفظ تنسيقك، ثم يتبعه في كل تذكرة مستقبلية.

3 • فئاتك ومنصاتك وحقوقك

ما عليك سوى إخبار BugIt بلغة طبيعية، وسيحدّث إعداداتك نيابةً عنك:

ما تكتبه (أمثلة)

```
My categories are Gameplay, UI, Audio, and Networking
We test on Windows and Xbox
```

4 • أدواتك الخاصة: الاتصالات المخصصة

هل تستخدم أداة غير موجودة في القائمة المدمجة؟ اربطها بالطريقة نفسها التي تربط بها أداة مدمجة:

ما تكتبه

```
Add a custom MCP server
```

امنحها اسمًا قصيرًا وعنوانها، الذي يجب أن يبدأ بـ `https://`، وسيُتصل بها BugIt ويتحقق من سلامتها نيابةً عنك.

5 • صحّح له ببساطة أثناء العمل

هذه أسهل طريقة على الإطلاق: عندما يُعدّ BugIt مسودة تذكرة، صحّح أي شيء لا يعجبك، سواء كان تسميةً أو صياغةً أو مستوى خطورة. عند تفعيل `learn from your edits`، وهو الخيار الموصى به، يتذكر تفضيلاتك ويطبّقها في المرة التالية. تتراكم معرفة مشروعك بصورة طبيعية، ولا يغادر أيّ منها جهازك أبدًا.

كل ما تضيفه ملكك ويبقى محليًا

يُحفظ مسردك وأسلوب الكتابة المعتمد لدى فريقك وفئاتك والتفضيلات التي تعلّمها BugIt على جهازك، وتُنشأ نسخة احتياطية منها قبل كل تحديث، ولا تُرسل إلى أي شخص مطلقًا.

6 • مشاركة إعدادك مع فريقك: ترخيص Team

بعد إعداد المسرد، وأسلوب الفريق، وخيارات نظام التتبع بالطريقة التي تريدها، امنح بقية فريقك الإعداد نفسه تمامًا بأمر واحد، بدلًا من تكرار الخطوات 1-3 على كل جهاز:

ما تشغله مرةً واحدة، بعد أن يصبح إعدادك كما تريده

```
python tools/share.py export
```

ينشئ ذلك الملف `config.share.zip`، الذي يجمع خيارات نظام التتبع وأداة الأعطال والاتصالات، والمسرد، وأسلوب الفريق، من دون أي كلمات مرور أو رموز وصول داخله. أرسله إلى فريقك.

ما يشغله كل زميل في الفريق

```
python tools/share.py import config.share.zip
```

يحصلون فورًا على مصطلحاتك الدقيقة، وأسلوب تقارير الخلل، وإعداد نظام التتبع؛ من دون إعادة كتابة الفئات أو إعادة لصق لقطة شاشة لأسلوب الفريق. ولا تتأثر تراخيصهم أو إعدادات أجهزتهم الخاصة.

إذا غيّر الاستيراد وجهة إرسال البيانات

إذا أضف إعداد مشترك اتصالاً مخصصاً جديداً أو عنواناً للإشعارات، يعرض BugIt بدقة ما تغيّر قبل حدوث أي شيء آخر؛ فلا يُطبّق التغيير بصمت. كما ينشئ أولاً لقطةً لإعداد زميلك الحالي، بحيث يمكن استخدام `Restore my settings` للتراجع عن عملية الاستيراد إذا لم تكن كما توقع.

الجزء 4 · التحديثات والنسخ الاحتياطية والسلامة

الحصول على التحديثات (بأمر واحد)

عندما يتوفر إصدار أحدث، يذكره BugIt مرة واحدة عندما تقول `hi`. تحديثات v1.x المجانية مشمولة ما دام ترخيصك ساريًا. لتثبيته:

أنت تكتب

Update

1. ينشئ BugIt أولاً نسخة احتياطية جديدة من ملفاتك.
2. ينزل الإصدار الجديد ويتحقق من أنه أصلي ولم يُعبث به (فهو موقع تشفيرًا، ويُرفض أي تحديث مزيف أو معدّل).
3. يثبتته من دون المساس بإعداداتك، أو مسرد مصطلحاتك، أو أسلوب فريقك، أو ترخيصك، أو رموز الوصول الخاصة بك.
4. يطلب منك إعادة تحميل VS Code (`Ctrl+Shift+P`) ← اكتب `Reload Window` ← `Enter`). ابدأ محادثة جديدة وستكون على الإصدار الجديد.

لا حذف للمجلدات، إطلاقًا

على خلاف التثبيت الأول، تتم التحديثات في مكانها. لا تحذف أي شيء أو تعيد تنزيله مطلقًا، وتظل تهيئتك محفوظة دائمًا ونُشأ لها نسخة احتياطية.

ما الذي يمكن تعديله يدويًا بأمان، وما الذي لا يمكن؟

آمن، ويُحتفظ به خلال كل تحديث: `config.json` ، و `github/instructions/house-style.instructions.md` . (راجع الجزء 3، فهذه هي الطريقة المدعومة لتخصيص السلوك)، و `github/glossary/terms.template.md` ، و `github/instructions/learned.instructions.md` ، و `vscode/mcp.json` .

غير آمن — لا تعدّل هذه الملفات يدويًا

ملف الوكيل (`github/agents/bugit-qa-agent.agent.md`)، وأي ملف آخر ضمن `github/instructions/` ، وأي شيء في `tools/` ، كلها أجزاء من المنتج نفسه وليست إعداداتك. **يستبدلها التحديث بصمت**، وعلى خلاف الملفات أعلاه، لا توجد نسخة احتياطية يمكنك استعادتها منها. يتحقق BugIt من ذلك عند بدء التشغيل ومرة أخرى قبل تثبيت أي تحديث مباشرة، ويحدّرك إذا وجد تغييرًا، لكن القاعدة الأكثر أمانًا هي ببساطة ألا تعدّلها.

النسخ الاحتياطية والاستعادة

ما يفعله

ما تكتبه

يحفظ لقطة محلية من إعداداتك، ومسرد مصطلحاتك، وأسلوب فريقك، وإعدادات نقاط نهاية MCP، وما تعلمه النظام. يُستبعد استحقاق جهازك الموقع وقيم بيانات الاعتماد.

Back up my settings

يعرض كل لقطة محفوظة مع تاريخها.

List backups

يعيد الحالة إلى لقطة محفوظة، وينشئ أولًا لقطة لحالتك الحالية، بحيث يمكن التراجع عن عملية الاستعادة نفسها.

Restore my settings

تلقائيًا قبل كل تحديث

ينشئ BugIt دائمًا نسخة احتياطية قبل تثبيت إصدار جديد، ولذلك لن يؤدي أي تحديث إلى فقدان إعداداتك. إذا بدا أي شيء غير صحيح، يعيده [Restore my settings](#) إلى وضعه السليم.

السلامة — ما الذي تحميه

 **dry run** = وضع المحاكاة

قل **dry run**، ولن تكتب أدوات Python المضمّنة في BugIt أي شيء، وسيرفض الوكيل عمليات الكتابة إلى نظام التتبع؛ وبذلك يقرأ فقط، ولا ينشئ تذاكر أو يرسلها، ولا يضيف تعليقات أو يرسل إشعارات. يعتمد هذا الرفض على تعليمات BugIt، لا على قفل تفرضه المنصة أو نظام التشغيل، لذلك استخدم بيانات اعتماد للقراءة فقط عند التقييم.

✓ لا شيء من دون موافقتك

تُعرض معاينة لكل تذكرة وتعليق؛ وأي إجراء لا يمكن التراجع عنه يتطلب أن يكون ردك هو **FILE IT** بالضبط، بوصفه رسالتك كاملة، ومرتبطة بالمسودة التي قرأتها للتو، وقابلًا للاستخدام مرة واحدة.

 **يعمل على جهازك**

لا يتواصل إلا مع الأدوات التي تربطها به ومع نموذج الذكاء الاصطناعي الخاص بك.

 **لا أسرار في الملفات**

تظل رموز الوصول في خزانة نظام التشغيل لديك، ولا توضع أبدًا في ملف نصي عادي، ولا تُرسل إلينا أبدًا.

بياناتك وخصوصيتك

الشيء الوحيد الذي يرسله BugIt إلى Taskivator هو بيانات الترخيص والتحديث: تسجيل الدخول إلى حساب BugIt الخاص بك (في المتصفح، على Portal)، ومعرف جهاز مجزأ أحادي الاتجاه (رمز مموّ لا يكشف هويتك)، واستحقاق **Solo** أو **Team** الذي توافق على استخدامه لهذا الجهاز. لا يوجد مفتاح ترخيص، وتظل كلمة مرورك في المتصفح؛ فلا تُدخل أبدًا في VS Code. لا تغادر تقارير الخلل، أو المواصفات، أو مسرد المصطلحات، أو الإعدادات، أو رموز الوصول لجهازك مطلقًا. لا قياس عن بُعد ولا تحليلات، أبدًا. ترد التفاصيل الكاملة في ملف **PRIVACY.md** داخل مجلد BugIt.

المخاطر والتنبيهات المهمة — يرجى قراءتها

قد يخطئ الذكاء الاصطناعي

يصوغ BugIt التقارير باستخدام نموذج ذكاء اصطناعي قد يسيء فهم التفاصيل أو يخلق شيئًا. اقرأ المعاينة دائمًا قبل أن تقول **yes**. فأنت توافق على ما سيُنشأ ويُرسل.

ينشئ التذاكر ويرسلها إلى نظام التتبع الحقيقي لديك

بمجرد موافقتك، تُنشأ تذكرة حقيقية. هل تتدرب؟ استخدم **dry run** لكتابة التقرير من دون إرساله.

سلامة مشروعك مسؤوليتك

تقع مسؤولية كيفية استخدامك BugIt وسلامة مشاريعك وبياناتك وأنظمة التتبع لديك عليك. تقلّل وسائل الحماية المخاطر، لكنها لا تحل محل مراجعتك.

ما لا يفعله BugIt

يتوفر تعيين للحقول جرى اختباره، مع ضبط درجة الخطورة/الألوية تلقائيًا، في Jira و Azure DevOps؛ ويستخدم Azure DevOps نسخة المعاينة العامة من MCP البعيد من Microsoft. يستخدم Confluence اتصال Atlassian الموجه نفسه. تُعد Sentry و GitHub و Linear و Notion تجريبية ويجري التحقق منها مباشرة باستخدام حسابك. يتصل كل شيء آخر عبر خادم MCP متوافق خاص بك، تعدّه بمساعدة BugIt. يستخدم النظام ذكاء الاصطناعي، سواء كان Copilot أو مفتاح Anthropic/OpenAI الخاص بك، وليس نموذجًا مضمّنًا. حجب البيانات الحساسة قائم على بذل أفضل جهد، ولا يعمل المنتج إلا في VS Code. ترخيص واحد = جهاز واحد لخطة Solo، أو ما يصل إلى 5 أعضاء أو مقاعد ترخيص للأجهزة لخطة Team. تعتمد النتائج واتساقها أيضًا على نموذج الذكاء الاصطناعي الذي يجب داخل Copilot؛ فالنموذج الأقوى يتبع خطوات السلامة بموثوقية أكبر من نموذج خفيف جدًا أو مجاني.

الجزء 5 - استكشاف الأخطاء وإصلاحها والأسئلة الشائعة

يمكن حل معظم العقبات خلال ثوانٍ، وغالبًا ما يكون أسرع حل هو أن تطلب من المساعد مباشرة.

🌟 جرّب هذا أولًا — اطلب من الذكاء الاصطناعي إصلاح المشكلة

كلما حدث خطأ، سواء أثناء الإعداد أو الاستخدام اليومي، يكون أسرع حل في الغالب هو سؤال المساعد مباشرة في المحادثة. صف ما حدث بكلمات بسيطة، مثل «توقف الإعداد في منتصفه» أو «لا يتصل جسر نظام التتبع الخاص بي»، واطلب منه إصلاحه. يستطيع الذكاء الاصطناعي تشخيص معظم المشكلات وإصلاحها في الحال. يرجى تجربة ذلك قبل مراسلتنا؛ فهو يحل الغالبية العظمى من المشكلات خلال ثوانٍ.

تطبيق إصلاح نقّذ الذكاء الاصطناعي: زر Keep

إذا أصلح المساعد شيئًا بتغيير ملف، يعرض VS Code شريطًا صغيرًا يحتوي على Keep / Undo. إذا كنت قد طلبت الإصلاح ورضيت عنه، فانقر على Keep؛ وسيطبّق التغيير على نسختك الخاصة فقط، على جهازك. انقر على Undo لتجاهله. لا تتم مشاركة أي شيء تختار له Keep مع أي شخص آخر.

ما ينفغي فعله	ما تراه
اكتب Activate. يفتح BugIt موقع BugIt Portal في متصفحك؛ سجّل الدخول، واختر استحقاق Solo أو Team، ووافق على هذا الجهاز. ألا تملك حسابًا بعد؟ احصل على حساب من المتجر.	"Activate to start"
اكتب Activate مرة أخرى لإعادة فتح BugIt Portal، ثم أكمل تسجيل الدخول والموافقة على هذا الجهاز. يتم التفعيل بالكامل في المتصفح؛ ولا يوجد مفتاح للصقه.	لم يُفتح المتصفح، أو لم يكتمل التفعيل
اكتب Begin setup واختر نظام التتبع لديك.	"No tracker enabled"
افتح .vscode/mcp.json وانقر على Start فوق الخادم. هل ما زالت المشكلة قائمة؟ اكتب fix servers واتبع الخطوات.	يظهر على الجسر رمز X أحمر، أو لا يتصل

يواصل مطابقتك بتسجيل الدخول	قل <code>fix servers</code> ، وأعد تشغيل الخادم، وأكمل تسجيل الدخول في المتصفح. لا يعرض BugIt أبدًا قيم بيانات الاعتماد المحفوظة. هل هذه هي المرة الأولى؟ راجع الخطوة 8c.
لا ينشئ تذكرة الخلل ولا يرسلها	اكتب <code>Check readiness</code> ؛ وسيعرض بالتحديد ما ينقصك، وغالبًا ما يكون السبب أن الجسر لم يبدأ أو أنك لم تسجل الدخول.
يبدو شيء ما معطلًا بعد تحديث	اكتب <code>Restore my settings</code> للتراجع.
يبدو أن الوكيل «لا يلتزم بالتعليمات»	اكتب <code>Read and follow all your given instructions</code> ، أو ابدأ محادثة جديدة. قد يحدث هذا بوتيرة أكبر مع نموذج ذكاء اصطناعي خفيف أو مجاني؛ ويكون النموذج الأقوى في متقي النماذج في Copilot أكثر اتساقًا.
تبدو الإجابات عامة أو غير مخصصة للمشروع	زوده بمزيد من السياق، أو اعرض عليه تذكرة حالية أثناء <code>Begin setup</code> ليتعلم أسلوبك. أعد تشغيل <code>Begin setup</code> لتحسينه.
تريد تغيير أحد خيارات الإعداد	اكتب <code>Begin setup</code> واذكر ما تريد تغييره، مثل <code>"add a tracker"</code> أو <code>"start over"</code> ؛ وسيعيد فتح ذلك الجزء فقط. إذا كان الإعداد قد انقطع في منتصفه فحسب، فإن كتابة <code>Begin setup</code> وحدها تستأنف من موضع التوقف بدلًا من البدء من جديد.

فحوص السلامة المضمّنة

`Check status` (هل أدواتي متصلة؟) · `Check readiness` (ما المتقي قبل أن أستطيع إنشاء تذكرة وإرسالها؟). لإجراء فحص أعمق، افتح `Terminal → New Terminal` وشغل `python tools/selftest.py`.

الأسئلة الشائعة

هل أحتاج إلى معرفة البرمجة؟

لا. إذا كنت تستطيع تثبيت تطبيق وكتابة جملة، فيمكنك استخدام BugIt. فهو يتولى الأجزاء التقنية نيابة عنك.

هل سينشئ تذاكر خلل ويرسلها من دون علمي؟

أبداً. تُعرض معاينة لكل تذكرة وتعليق أولاً، ولا يحدث أي إجراء لا يمكن التراجع عنه إلا عندما يكون ردك هو `FILE IT` بالضبط. تتم مطابقته بوصفه رسالتك كاملة، ولذلك لا يترتب أي إجراء على جملة تُرد فيها العبارة عرضًا. وهو مرتبط بالمسودة نفسها التي قرأتها، ولذلك يطلب BugIt موافقتك من جديد إذا حدث تغيير لاحق. كما يُستخدم مرة واحدة، فلا يمكن لأمر مكرر أن ينشئ التذكرة ويرسلها مرتين. لا يستطيع BugIt أبدًا حذف تذكرة، أو تغيير حالتها، أو تغيير المكلف بها. للتدرب، قل `dry run` ؛ فلن تكتب أدوات Python المضمّنة في BugIt أي شيء، وسيرفض الوكيل عمليات الكتابة إلى نظام التتبع، مع أن هذا الرفض يعتمد على تعليمات BugIt لا على قفل تفرضه المنصة أو نظام التشغيل، لذلك استخدم بيانات اعتماد للقراءة فقط عند التقييم.

هل يجب أن أبدأ الجسر في كل مرة؟

نعم. افتح `.vscode/mcp.json`. وانقر على `Start` عندما تعيد فتح VS Code. يتذكر النظام بيانات تسجيل الدخول المحفوظة؛ ولا يحتاج إلى إعادة التشغيل سوى الجسر.

هل يمكنني استخدامه على حاسوبي؟

يمكن استخدام جهاز واحد في كل مرة مع **Solo** ، بينما تغطي **Team** ما يصل إلى 5 أعضاء أو مقاعد ترخيص للأجهزة، بمقعد واحد لكل عضو. لا بأس بالانتقال إلى جهاز جديد: نشط الجهاز الجديد، وسيتوقف الجهاز القديم فورًا عن القدرة على تجديد تفويضه. يتحرر مقعده بمجرد أن يعيد الجهاز القديم الاتصال، مؤكدًا أنه تخلى عن الوصول، أو عند انتهاء فترة التفويض الحالية للعمل دون اتصال البالغة 72 ساعة؛ ولذلك لا توجد فترة انتظار ثابتة. اكتب **License status** للتحقق.

ماذا يحدث إذا تعطل خادم الترخيص؟

يمكنك مواصلة العمل. بعد تفعيل ناجح عبر الإنترنت، يعمل BugIt بموجب تفويض مخزن محليًا لمدة تصل إلى 72 ساعة من دون اتصال، ولذلك لن تقاطعك أعطال الخادم، أو حتى عطلة نهاية أسبوع بلا إنترنت، خلال تلك المدة. بعد ذلك يعيد التحقق عبر الإنترنت.

هل أحتاج إلى GitHub Copilot؟

إنه أسهل طريقة. إذا لم يكن متوفرًا لديك، شغل **python tools/setup_wizard.py** في **Terminal** لإجراء الإعداد، ويمكن لBugIt العمل بصورة مستقلة باستخدام مفتاح Anthropic/OpenAI الخاص بك عبر **python standalone/run.py**.

هل بياناتي خاصة؟

نعم. الشيء الوحيد الذي يرسله BugIt إلى Taskivator هو بيانات الترخيص والتحديث: تسجيل الدخول إلى حساب BugIt الخاص بك في المتصفح على Portal، ومعرّف جهاز مجزأ، واستحقاق **Solo** أو **Team** الذي توافق عليه لهذا الجهاز. لا يوجد أي قياس عن بُعد، أبدًا. لا تذهب تذاكرك نفسها إلا إلى نموذج الذكاء الاصطناعي ونظام التتبع اللذين تربطهما، ولا تصل إلينا أبدًا.

هل يمكنني تعديل ملفات BugIt؟

يمكنك تخصيص مسرد المصطلحات وأسلوب الكتابة المعتمد لدى فريقك والإعدادات الواردة في الجزء 3. لكن لا تعطل نظام الترخيص أو التفعيل. إذا واجهت خللاً حقيقيًا في الأداة نفسها، فراسل الدعم حتى يمكن إصلاحه للجميع في التحديث التالي.

مع أي أنظمة تتبّع العيوب يعمل؟

يحصل Jira و Azure DevOps على إعداد موجه مع تعيين للحقول جرى اختباره، وضبط تلقائي لدرجة الخطورة/الأولوية؛ إذ يستخدم Atlassian Rovo MCP Jira، ويستخدم Azure DevOps نسخة المعاينة العامة من MCP البعيد من Microsoft. تستخدم Sentry و GitHub و Linear و Notion نقطة نهاية معروفة يعدّها BugIt ويتحقق منها مباشرة باستخدام حسابك، وتظل تجريبية حتى تجتاز تلك الفحوص. تتصل GitLab و Shortcut و YouTrack و Bugzilla و ClickUp و Asana و Trello عبر خادم MCP متوافق خاص بك. يصوغ BugIt التذكرة ثم ينشئها ويرسلها عبر الموصّل الذي تفعله. اختر نظام التتبع لديك أثناء الإعداد، وبدّله في أي وقت باستخدام **Begin setup**.

هل سيكتشف تقارير الخلل المكررة قبل أن أرسلها؟

نعم. قبل كتابة أي شيء، يبحث في نظام التتبع لديك عن تقارير مشابهة، حتى إن كانت صياغتها مختلفة قليلًا، ويعرض لك النتائج المطابقة كي تتجنب تسجيل الخطأ نفسه مرتين. ويظل قرار المتابعة لك.

هل يمكنه كتابة التذاكر بأكثر من لغة؟

نعم. أضف لغة ثانية أثناء الإعداد، وسيكتب كل تقرير باللغتين في كتلتين منفصلتين، مع استخدام مسرد مصطلحاتك كي تظل مصطلحات المنتج دقيقة؛ فهو لا يرتجل ترجمة أبدًا.

هل يمكنني إرفاق لقطة شاشة؟

ألصق الصورة فحسب في المحادثة. يقرأها BugIt، ويحجب أي معلومات حساسة يلاحظها، ويرفقاها بالتذكرة بعد موافقتك.

هل يخفي كلمات المرور ورموز الوصول في تقاريري؟

عند تشغيل حجب البيانات الحساسة، يزيل عناوين البريد الإلكتروني ورموز الوصول وعناوين IP من كل مسودة قبل إرسالها. وترى دائماً النص الكامل في المعاينة أولاً.

أنشأت تذكرة وأرسلتها عن طريق الخطأ؛ هل يمكنني التراجع عنها؟

تمنع المعاينة وكتابة **yes** معظم الأخطاء قبل كتابة أي شيء. إذا كنت قد فعلت سجل التدقيق، فإكتب **Undo last filing**. وإلا، فأغلق التذكرة أو عدّلها في نظام التتبع لديك كالمعتاد.

هل يمكنه مساعدتي في التحقق من تقرير خلل أو إغلاقه؟

نعم. اطلب تعليق تحقق، وسيصوغ تعليقاً واضحاً يبيّن نجاح التحقق أو فشله، بلغاتك، على التذكرة الحالية؛ وتوافق عليه قبل نشره.

هل يمكنني استخدامه لأكثر من مشروع؟

يُعدّ كل مجلد BugIt للعمل مع نظام تتبّع واحد لمشروع واحد. لمشروع آخر، إما أن تعيد تشغيل **Begin setup** لتوجيهه إلى مكان جديد، أو تحتفظ بنسخة منفصلة لكل مشروع.

كيف أحصل على التحديثات؟

عند صدور إصدار جديد، يذكره BugIt عند بدء المحادثة. اكتب **update** وسيُنبّه في مكانه؛ مع الاحتفاظ بمسرد مصطلحاتك، وأسلوب فريقك، وإعداداتك، وإنشاء نسخة احتياطية منها أولاً.

ماذا يحدث عند انتهاء ترخيصي؟

عند انتهاء مدتك، جُدّ من حسابك؛ وسيطبّق جهازك الاستحقاق المجدّد في المرة التالية التي يتحقق فيها عبر الإنترنت. فترة التفويض للعمل دون اتصال البالغة 72 ساعة والمذكورة أعلاه منفصلة؛ فهي لا تغطي إلا انقطاعاً مؤقتاً/خادم الترخيص، ولا تغطي انتهاء مدة الترخيص. اكتب **License status** في أي وقت لمعرفة وضعك.

إذا جددت ترخيصي أو اشتريت ترخيصاً آخر، فهل تتراكم الأيام؟

لا، لا تتراكم التراخيص. يحل التجديد محل استحقاقك الحالي، وتبدأ مدته من جديد من يوم سريانه، ولا تُرخل الأيام غير المستخدمة. لذلك جُدّ قرب نهاية مدتك الحالية، لا مبكراً.

الجزء 6 - الترخيص والدعم

شروط الترخيص (ملخص بلغة مبسطة)

ترد الشروط الكاملة والملزمة في ملف **LICENSE** داخل مجلد BugIt، وتلك الوثيقة هي الحاكمة. باختصار:

الموضوع	بلغة مبسطة
مرخص، وليس مبيعاً	تشتري ترخيصاً لاستخدام BugIt، ولا تشتري ملكيته. تحتفظ Taskivator بجميع الحقوق.
مقاعدك	Solo = جهاز واحد في كل مرة؛ و Team = ما يصل إلى 5 أعضاء أو مقاعد ترخيص للأجهزة. حسابك واستحقاقك خاصان بك؛ فلا تشاركهما أو تبيعهما من جديد أو تنشرهما.
إلغاء الصلاحية	يمكن إلغاء الوصول إذا تمت مشاركته، أو رُدّ ثمن الترخيص، أو أُلغيت الدفعة باعتراض مصرفي، أو أُسيء استخدامه.
خصّصه، ولكن...	عدّل إعداداتك، ومسرد مصطلحاتك، وقوالبك بحرية، لكن لا تعطل نظام الترخيص أو التفعيل ولا تتحايل عليهما.

مسؤوليتك	تقع مسؤولية كيفية استخدامك BugIt وسلامة مشروعك عليك. تراجع كل تقرير وتوافق عليه قبل إنشاء التذكرة وإرسالها. وسائل الحماية أدوات مساعدة وليست ضمانات.
«كما هو»، والمسؤولية	يُقدّم كما هو، من دون ضمان. وفي حدود ما يسمح به القانون، تقتصر المسؤولية على المبلغ الذي دفعته، مع استبعاد الأضرار غير المباشرة أو التبعية.
المدة والقانون	ترخيص لمدة سنة واحدة تبدأ في يوم تفعيله؛ وهو عملية شراء لمرة واحدة من دون تجديد تلقائي، وتحكمه قوانين اليابان. تُعالج عمليات استرداد الأموال من خلال المتجر الذي اشتريت منه.
التجديدات لا تتراكم	يحل التجديد محل استحقاقك الحالي؛ وتبدأ مدته من جديد، ولا تُرخل الأيام غير المستخدمة، لذلك جدّد قرب نهاية مدتك. عند انتهاء المدة، جدّد من حسابك، وسيُعاد تفويض جهازك عند اتصاله التالي بالإنترنت للتحقق.

وثيقتان في مجلدك

LICENSE (الشروط الكاملة) و **PRIVACY.md** (بيان الخصوصية). بتنشيط BugIt واستخدامه، فإنك تقبل **LICENSE**.

الحصول على المساعدة

يرجى مراجعة قسمي استكشاف الأخطاء وإصلاحها والأسئلة الشائعة أعلاه أولاً؛ فهما يغطيان كل شيء تقريباً. ثم اتبع هذا الترتيب:

1 • اطلب من الذكاء الاصطناعي إصلاح المشكلة ★

أفضل خطوة أولى: صف المشكلة في المحادثة واطلب من المساعد إصلاحها. إذا غيّر ملفاً وبدأ التغيير صحيحاً، فانقر على **Keep** لتطبيقه على حاسوبك فقط.

2 • شغل فحصاً للحالة

Check status · **Check readiness** · أو **python** **tools/selftest.py** في **Terminal**.

3 • الاستعادة

يلغي **Restore my settings** تغييراً أو تحديثاً سيئاً.

4 • تواصل مع شخص

فقط إذا لم يساعدك الدليل والذكاء الاصطناعي: زُر **bugit.dev** أو أرسل بريداً إلكترونيّاً إلى **support@bugit.dev**، علماً بأن الدعم يُقدّم باللغة الإنجليزية. هل تواجه مشكلة في الإعداد خلال أيامك السبعة الأولى؟ سنساعدك على تشغيله أو نساعدك في استرداد المبلغ من المتجر.

يرجى عدم تضمين معلومات المشروع السرية في رسائل الدعم

يختلف كل مشروع عن الآخر، وقد يكون قدر كبير من عملك سرياً. إذا أرسلت إلينا بريداً إلكترونيّاً، فيرجى وصف مشكلة BugIt بعبارات عامة فقط؛ ولا تلمص شيفرات سرية، أو تفاصيل تقارير أخطاء، أو مواصفات، أو لقطات شاشة، أو بيانات من مشروعك. لا تتحمل Taskivator مسؤولية أي معلومات سرية تُرسل إلينا، ويُحذف فوراً أي محتوى سري نتلقاه. كلما أمكن، اطلب المساعدة من مساعد الذكاء الاصطناعي أولاً؛ فهو يستطيع حل معظم المشكلات مباشرة داخل محررك.

شكراً لاختيارك BugIt.

أنشئ تذاكر خلل واضحة، وتجنّب التكرارات، واستعد وقتك؛ خللاً تلو الآخر.